LATTICRA

The Latticra System Substrate

An Effect at Modern Security

A computer scientist's guide to evidence-bound, effect-aware project security

Evidence-first architecture for local integrity, policy boundaries, reproducible reports, and future runtime-handoff research.

Working Draft 0.1 - 2026-05-24

Front Matter

This book overhauls the original Latticra Seal handbook into a project-level explanation of Latticra as a system substrate. It is written for computer scientists: readers who care about invariants, authority boundaries, reproducible evidence, policy semantics, and the difference between measuring a system and enforcing a system.

The project should be understood as early and evidence-bound. The text deliberately avoids calling Latticra a production security product, a host-protection system, a sandbox, a kernel enforcement layer, a ransomware prevention system, or a malware prevention system. The value proposition presented here is narrower and more interesting: Latticra is a substrate for making security-relevant effects visible, typed, reviewable, and eventually governable.

Source posture: the source material available for this edition included the uploaded Seal handbook and recent project evidence around the Latticra Panel installer, guarded user-local installation, receipts, component markers, Fedora validation traces, and a one-pass field-engine browser artifact. Where the broader project is not fully specified by those materials, this book defines a conservative model rather than asserting unverified implementation details.

Contents

Part I - Orientation

Latticra begins as a way to make system effects legible before it tries to make them enforceable.

Chapter 1. Why Latticra Exists

Modern systems fail as much from invisible authority and ambiguous claims as they do from isolated bugs. Latticra treats security posture as something that must be measured, constrained, documented, and reproduced before it is trusted.

Most security projects begin by promising protection. Latticra should begin with a more careful claim: it can help a developer and operator know what the project says it did, what evidence it can produce, and which authorities it explicitly did not use. This is not a weaker story. It is a computer-science story. It turns vague trust into state, transition, policy, and evidence.

The original Seal handbook presents Seal as the verification, reporting, and policy-boundary layer inside the ecosystem. This book expands that layer into a whole-project architecture: Latticra is a substrate whose primary job is to convert local project state into inspectable evidence and to keep unsupported authority claims from crossing a boundary unnoticed.

The phrase "an effect at modern security" is used here as a technical metaphor. In programming-language terms, an effect is something a computation does outside pure value production. Reading files, writing receipts, mutating host state, requesting network authority, installing wrappers, modifying kernel surfaces, and claiming runtime enforcement are effects. Latticra is useful when it makes those effects explicit and typed.

The project therefore addresses a recurring security engineering problem: a tool may be locally helpful, but nobody can quickly tell which authority it had, which state it observed, which files it changed, whether its report is reproducible, and whether the words around it outrun the tests. Latticra answers by making the default path evidence-first and claim-limited.

1.1 The central thesis

Latticra is an evidence-bound system substrate for describing, measuring, and presenting local project state under explicit authority constraints. It is not currently a production host defense product. Its importance is the discipline it introduces: before a project can enforce a security boundary, it must be able to name the boundary, test the boundary, report the boundary, and deny claims that exceed it.

- Evidence before enforcement: reports and receipts must precede security claims.
- Authority before mutation: root, network, runtime, kernel, systemd, and SELinux authority must be explicit.
- Policy before pass: a PASS means declared local expectations were met, not that the host is secure.
- Documentation before marketing: claims are bounded by implemented, tested, and reproducible behavior.

1.2 The reader model

This book assumes the reader is comfortable with operating systems, local package layouts, CI lanes, hashing, manifests, policy checks, and basic formal modeling. The goal is not to teach security from first principles. The goal is to explain the shape of Latticra as an artifact that computer scientists can inspect, criticize, extend, and test.

1.3 The smallest useful mental model

Figure 1.1 - Minimal Latticra model.

```
Project tree P
  -> declared scope M
  -> measured state H
  -> policy decision D
  -> report or receipt R
  -> claim set C

Latticra is useful when:  C <= evidence(R) <= test(D) <= implemented effects(P)
```

The model says that public claims must be no stronger than evidence. Evidence must be no stronger than tests. Tests must be no stronger than implemented effects. This creates a ladder that prevents the project from jumping directly from "we computed a hash" to "we secure the host."

Chapter 2. The Substrate Model

A substrate is a layer that other parts of a system can stand on. In Latticra, the substrate is not an operating system replacement; it is a vocabulary and evidence pipeline for system state, authority, and future security decisions.

The substrate idea is intentionally narrower than a platform claim. A platform normally promises user-facing capabilities. A substrate promises stable underlying relations: what can be observed, what can be recorded, what can be denied, and what future layers may consume. The Latticra substrate is a set of local mechanisms and design rules that encourage every meaningful effect to leave behind an intelligible witness.

2.1 Substrate objects

Object	Role in the substrate	Current posture
Manifest	Declares what the tool may inspect and what expectations apply.	Core Seal concept; schema should be hardened.
Lock/baseline	Captures hash material or expected state against which change is measured.	Present as hash lock material; stronger signed evidence is future work.
Policy	Accepts or denies state and authority requests.	Conservative; should fail closed.
Report	Human-readable and eventually machine-readable description of checks.	Report-only lane is central.
Receipt	Operator-facing record of installation or action.	Used by local installer evidence.
Panel	Human interface for plans, evidence, and guarded local actions.	Bridge/control surface, not a root authority.
Runtime handoff	Future metadata path for enforcement decisions.	Planned/research; not enforcement today.

These objects are not just files. They are an attempt to structure trust. A manifest limits scope. A lock makes drift visible. A policy denies unsupported states. A report tells the operator what happened. A receipt records what was installed or would have been installed. The Panel makes these relations visible instead of hiding them behind a button.

2.2 Security as an effect system

For a computer scientist, the natural abstraction is an effect system. A computation that only reads files has a different type from one that writes to a user prefix. A tool that writes receipts has a different type from one that modifies systemd. A UI that previews an install plan has a different type from one that uses root to mutate the host. Latticra can be described as an attempt to classify those effects and reject mismatches.

Effect class	Examples	Allowed current interpretation
ReadLocal	Read manifest, inspect project tree, read local config.	Allowed under declared scope.
MeasureLocal	Hash files, compare lock baseline, count changes.	Allowed when report-only and read-only.
WriteEvidence	Write report, plan, receipt, component marker.	Allowed when documented and local.
UserPrefixMutation	Create files under a guarded user-local prefix.	Allowed only by guarded installer profile.
NetworkOperation	Download, publish, call remote APIs.	Not a current authority lane.
RootMutation	Install as root, modify /usr, services, kernel, SELinux.	Not a current authority lane.
RuntimeEnforcement	Sandbox, block process, enforce isolation.	Future research; not currently claimed.

2.3 Why this matters

Security systems often degrade when documentation, implementation, and authority drift apart. Developers add flags. Installers add convenience. UI buttons hide side effects. CI proves a narrow property while README text promises a broad one. Latticra can become valuable because it treats that drift as the primary enemy. The substrate exists to keep claims, tests, and effects synchronized.

Chapter 3. Current Evidence and Claim Ledger

The strongest part of the project is not a large production guarantee. It is the explicit ledger of what exists, what is planned, and what must not be claimed yet.

The Seal handbook states a current readiness posture: local report generation, manifest/hash baseline, policy regression, and Panel bridge planning are positive claims. Runtime enforcement, network operation, root installation, kernel enforcement, and production readiness are zero claims. That ledger should not be a footnote. It should be the core of the system.

Capability or claim	Status in this edition	How to talk about it
Local report generation	Current evidence-bound capability	Seal can generate local evidence reports.
Manifest/hash baseline	Current evidence-bound capability	Seal can compare local evidence against declared expectations.
Policy regression lane	Current evidence-bound capability	Denied states are part of the test story.
Panel bridge	Planning plus installer/control-surface evidence	Panel should surface reports and authority state safely.
Guarded local-prefix installer	Evidence from local installer work	Installer can operate as user-local, guarded, and receipt-generating.
Runtime enforcement	Not current	Future handoff metadata only until enforcement exists and is tested.
Network authority	Not current	Do not claim network authority.
Root authority	Not current	Do not claim root installation.
Kernel/systemd/SELinux integration	Not current	Research or future integration only.
Production host security	Not current	Do not claim host protection, malware prevention, or ransomware prevention.

3.1 Claim discipline

A useful project book must not launder aspiration into implementation. For Latticra, aspiration is important: future cryptographic receipts, runtime-boundary metadata, and Linux integration research are all plausible directions. But they must remain future directions until a reader can reproduce the behavior.

The claim ledger should be kept in the repository as a first-class artifact. Every README, installer panel, report header, and public project page should be checked against it. When a capability moves from zero to one, the transition should require implementation, tests, documentation, and a short rationale.

3.2 The invariant behind the ledger

Algorithm 3.1 - Claim gate invariant.

```
For every public claim c in ClaimSet:
    require exists evidence e
    require exists test t
    require t reproduces e
    require implementation effect supports c
```

```
    otherwise deny c or mark c as future work
```

This invariant is stronger than a checklist. It is a design philosophy that turns documentation into an enforceable development constraint. The Latticra project should treat exaggerated language as a failing test of the public interface.

Part II - Project Architecture

The project is a collection of lanes: Seal, CLI contracts, Panel, installer, validation, and research artifacts. Their shared job is to preserve authority boundaries.

Chapter 4. Latticra Components

Latticra is best explained as a system of cooperating components rather than one monolithic security product.

The available project evidence exposes several component names and workflows: Latticra Seal, Latticra CLI wrappers, Lat tooling, LIR contracts, the Latticra Panel, a guarded local-prefix installer, docs/examples, developer CLI helpers, Fedora/RPM validation lanes, and demonstration artifacts such as the one-pass field engine. This book treats those as separate lanes in a single substrate.

Component lane	Primary interface	Substrate responsibility
Seal	latticra-seal, reports, manifests, locks	Describe, measure, compare, deny unsupported claims, emit local evidence.
CLI / wrappers	latticra, lat, latticra-seal, latticra-panel, installer commands	Provide stable entry points and command contracts.
Lat tooling	lat command and related installed components	Project tooling lane; details should be specified by contracts.
LIR contracts	Installed contract docs and validation lanes	Intermediate representation or contract surface; should remain test-driven.
Panel	Graphical local installer/control surface	Expose plan, evidence, authority, and guarded actions to the operator.
Installer	dry-run, local-example, verify-local, uninstall-local	Build or install user-local payloads and receipts with explicit authority limits.
Validation	make targets, CI, Fedora VM/RPM lanes	Prove bounded claims and record non-claims.
Research artifacts	one-pass field engine and visual demos	Demonstrate concepts without claiming security boundaries.

4.1 The architecture as dataflow

Figure 4.1 - Latticra lanes as a conservative dataflow.

```
Repository and local prefix
    |
    +--> CLI wrappers and developer tools
    +--> Seal manifest and lock material
    +--> Panel and installer configuration
    +--> docs, examples, receipts, validation transcripts

Seal lane:
    source tree -> manifest/config -> hash baseline -> policy -> report

Installer lane:
    config -> plan -> guarded prefix mutation or dry-run -> receipt -> verification

Panel lane:
    profile -> authority view -> plan/evidence -> guarded action -> status
```

The design becomes easier to reason about when each lane is interpreted by its effects. The Seal lane should mostly read and report. The installer lane may write inside a guarded user-local prefix. The Panel lane may make plans and evidence visible, but it should not silently upgrade itself into root, network, or runtime authority.

4.2 Component seams

The most important design problem is the seam between components. The Seal report may be shown in the Panel. The installer may write a Seal config. CLI wrappers may call Seal. Validation lanes may assert that those calls are safe. Each seam should have an interface contract that states inputs, outputs, authorities, errors, and proof artifacts.

- Seal-to-Panel seam: Panel may invoke report-only checks and display reports; it must not grant enforcement authority.
- Installer-to-Seal seam: installer may install report-only Seal configuration and component markers; it must record receipts.
- CLI-to-Policy seam: every user-facing command must document read effects, write effects, network use, root use, PASS, and FAIL.
- Validation-to-README seam: CI results should constrain public wording and prevent stale status claims.

Chapter 5. Latticra Seal

Seal is the project layer that turns local state into bounded evidence. It is the original center of the handbook and remains the most explicit subsystem.

Seal is not a separate production security product. It is a Latticra substructure for evidence, reporting, policy boundaries, manifests, hash baselines, and future runtime-boundary metadata. Its current intellectual strength is restraint: it begins with describing what exists, measuring what exists, reporting what changed, and refusing to claim authority it does not have.

5.1 The Seal pipeline

Figure 5.1 - Seal pipeline.

```
source tree
  -> manifest / seal config
  -> hash baseline
  -> policy checks
  -> report-only evidence
  -> Panel / CLI / future runtime-boundary handoff
```

5.2 Manifest layer

The manifest layer defines the legal scope of inspection. A mature manifest should specify included paths, excluded paths, expected files, policy settings, report behavior, and authority declarations. A malformed manifest should not be interpreted generously; it should produce a denial that a developer can fix.

Example 5.1 - Manifest shape for scoped local evidence.

```
# illustrative latticra.seal manifest - not a normative schema
project = "Latticra"
mode = "report-only"

[scope]
include = ["docs", "installer", "examples", "fixtures"]
exclude = [".git", "target", "build", "*.tmp"]

[authority]
network = false
root = false
runtime_enforcement = false
kernel_modification = false
systemd_modification = false
selinux_modification = false

[policy]
fail_closed = true
deny_outside_scope = true
deny_unknown_authority = true
```

5.3 Measurement layer

The measurement layer computes local evidence. In its conservative mode it should read files, compute hashes, compare against the saved baseline, report changed/missing/new files, and avoid mutation unless explicitly running a baseline update command. The measurement layer is not a judge by itself; it produces facts for policy and reports.

Algorithm 5.1 - Read-only measurement skeleton.

```
def measure(manifest, lock):
    assert manifest.mode in {"report-only", "verification"}
    paths = resolve_scope(manifest.scope)
    evidence = []
    for path in paths:
        digest = hash_file(path)
        expected = lock.digest_for(path)
        evidence.append(compare(path, digest, expected))
    return evidence
```

5.4 Policy layer

The policy layer decides whether the measured state is acceptable. The default must be conservative. Unsupported authority claims, malformed manifests, denied paths, and missing metadata should not pass. Policy should be understood as a compiler from messy project state into explicit pass/fail/deny outcomes.

5.5 Report layer

The report layer emits human-readable and eventually machine-readable evidence. The source handbook identifies fields such as timestamp, mode, prefix, kernel, OS, authority flags, manifest/lock presence, hash status, warnings, failures, and final status. A report-only PASS means the checked evidence matched local expectations for that command. It must not be interpreted as proof that the host is secure.

Example 5.2 - Report semantics, with explicit non-claim.

```
timestamp_utc=2026-05-24T00:00:00Z
mode=report-only
prefix=/home/user/.local/share/latticra
network_authority=0
runtime_enforcement_authority=0
root_authority=0
manifest_present=1
lock_present=1
hashes_checked=128
hashes_changed=0
warnings=0
failures=0
status=PASS
meaning=local evidence matched declared expectations; host security not claimed
```

Chapter 6. Panel, Installer, and Receipts

The Panel and installer are the human-facing path into the substrate. Their job is to make plans, effects, and authority visible before anything effectful happens.

The broader project evidence shows Latticra Panel as a graphical local installer and control panel for Latticra, Lat, LIR, and Latticra Seal. It includes workflows for safe dry-run, guarded user-local installation, verification, and launch. This should be interpreted as a local developer/operator interface, not as a host security authority.

6.1 Why a Panel matters

A command-line tool can document authority, but a graphical installer can either clarify or obscure it. Latticra Panel should be designed to clarify. It should make authority visible in the UI, show whether the current action is dry-run or guarded local-prefix install, surface the plan and receipt paths, and keep network/root/runtime authority disabled unless future work explicitly implements and tests a controlled lane.

Panel action	Expected effect	Boundary
Save configuration	Write local installer config.	Local file write; no host-wide mutation.
Generate plan	Write or refresh install plan.	Evidence output.
Run dry-install	Validate configuration and write dry-run receipt.	No host mutation.
Install guarded local prefix	Write under allowed user-local prefix.	No root/system mutation; receipt required.
Open evidence	Show plan/report/receipt material.	Read/display only.
Change Seal profile	Select intended crypto/report posture.	No signing/encryption unless implemented and tested.

6.2 Guarded local-prefix installation

The installer lane is a concrete example of Latticra thinking. A dry-run can generate a plan and receipt without mutating the host. A guarded local-prefix installation can write under a user-local prefix and install wrappers, docs, examples, component markers, and receipts. It should refuse root authority, network authority, and runtime enforcement authority in the current stage.

Example 6.1 - Current safe installer workflow shape.

```
make -C installer dry-run
make -C installer local-example
make -C installer verify-local
make -C installer uninstall-local

# Expected current posture:
# - user-local prefix
# - no root authority
# - no network authority
# - no kernel/systemd/SELinux modification
# - operator receipt required
```

6.3 Receipts as substrate objects

A receipt is not merely a log. It is a structured witness that an effect happened or would have happened. In Latticra, receipts should say when an action occurred, which mode was used, which prefix was targeted, which plan/config was used, which paths were written, and which authorities were not used.

Receipt field	Reason
timestamp_utc	Makes the action temporally reproducible.
mode	Distinguishes dry-run from local-prefix install.
result	Records success, failure, or denial.
install_prefix	Defines the write boundary.
config and plan	Links decision input to effect output.
installed_paths	Shows exactly which local areas changed.
authority flags	Prevents hidden root/network/runtime assumptions.

Receipts are how Latticra makes installation security a data problem. Instead of asking whether an installer is trusted in the abstract, the reader asks which effects it recorded and whether those effects fit the declared authority vector.

Chapter 7. Lat, LIR, and Contract Surfaces

The available evidence names Lat tooling and LIR contracts as installed components. This edition treats them as contract surfaces until fuller language or IR specifications are available.

A project book should include Lat and LIR, but it should not invent semantics that the available documents do not prove. The disciplined phrasing is: Lat tooling and LIR contracts appear as Latticra component lanes, installed alongside Seal, docs/examples, and developer CLI helpers. Their project role should be formalized through interface contracts, examples, and validation tests.

7.1 Contract-first explanation

The most useful way to present Lat and LIR today is as names for a language/tooling lane and an intermediate-representation or contract lane. A future edition can replace this conservative explanation with a full specification. For now, the important architectural claim is that they should obey the same Latticra substrate rules as Seal and the installer: declare effects, emit evidence, fail closed on unsupported authority, and keep public claims tied to tests.

Surface	Contract questions to answer
lat command	What inputs does it read? What files does it write? What exit codes matter?
LIR artifacts	What is the canonical syntax? What invariants are checked? What is lowered or compiled?
Lat-to-LIR lane	What transformations are allowed? What witnesses are emitted?
Runtime boundary metadata	Which LIR facts can become runtime decisions, and under what future authority?
Tests	Which examples prove stable behavior and which are aspirational?

7.2 Suggested formal shape

Definition 7.1 - Contract skeleton for Lat, LIR, and CLI surfaces.

```
CommandContract = {
    name: String,
    reads: Set<PathOrResource>,
    writes: Set<PathOrResource>,
    authority: AuthorityVector,
    inputs: Schema,
    outputs: Schema,
    pass_condition: Predicate,
    fail_condition: Predicate,
    denial_condition: Predicate,
    receipt_or_report: Optional[Schema]
}
```

The key is not the exact type syntax. The key is the discipline: a Latticra command should not be considered specified until a reader can identify its read set, write set, authority vector, output evidence, and failure modes.

Chapter 8. Field Engine and Research Artifacts

The one-pass field engine is best understood as a research and communication artifact: a visual state lattice with exportable witness material, not a standalone security boundary.

The one-pass field engine material describes a single-file browser demo with deterministic seed behavior, visual state metrics, hashing when browser crypto is available, and an exportable proof manifest. The artifact is useful because it communicates the Latticra aesthetic: state, coherence, entropy, seal, snapshots, and exportable evidence. Its own notes say it is not a standalone security boundary.

8.1 Why include demos in a security substrate?

Security research projects often need artifacts that teach the mental model before the implementation is complete. A visual field engine can make the substrate idea tangible. It shows state evolving, stabilization toggles, snapshots, and exported evidence. But the book must preserve the boundary: a demo can illustrate witness generation without becoming host protection.

Artifact behavior	Substrate lesson
Deterministic seed	Reproducibility matters.
Observable node/edge state	State should be inspectable.
Entropy and coherence metrics	Derived signals can summarize state, but need definitions.
SHA-256 when available	Hashing can bind a witness to observable state.
Exportable manifest	Evidence should be portable.
Explicit non-security note	Demonstrations must not overclaim.

8.2 From demo to research method

The general lesson is that every Latticra artifact should produce a witness. A browser demo exports a manifest. A Seal run emits a report. An installer emits a receipt. A validation lane emits a transcript. A future runtime handoff should emit a decision trace. The forms differ, but the substrate pattern remains stable.

Part III - Security Model

Latticra makes sense when security is modeled as typed effects, explicit authority vectors, and fail-closed policy decisions.

Chapter 9. Authority Vectors and Effect Classes

The authority vector is the core type of the Latticra system substrate. It records which powers a command has and, equally important, which powers it does not have.

An authority vector should be attached to reports, receipts, plans, UI states, and command contracts. It should be boring, explicit, and repeated everywhere. Repetition is a feature: a user should not have to infer whether root, network, runtime, kernel, systemd, or SELinux authority was used.

Definition 9.1 - Current authority vector shape.

```
AuthorityVector = {
    network_authority: Bool,
    runtime_enforcement_authority: Bool,
    root_authority: Bool,
    kernel_modification_performed: Bool,
    systemd_modification_performed: Bool,
    selinux_modification_performed: Bool,
    production_security_product: Bool
}
```

9.1 Deny by type mismatch

If a command is typed as report-only and attempts a network operation, that is an effect mismatch. If an installer profile has root_authority=false and attempts to write outside the user-local prefix, that is a boundary violation. If a README claims runtime enforcement but the authority vector says runtime_enforcement_authority=0, that is a claim violation. The same model can catch code problems and documentation problems.

Algorithm 9.1 - Authority gate.

```
def authority_gate(requested, allowed):
    for k, v in requested.items():
        if v and not allowed.get(k, False):
            return Deny(reason=f"unsupported authority: {k}")
    return Pass()
```

9.2 Effect taxonomy

The effect taxonomy lets developers reason about commands before implementation. A command that reads a manifest and prints a report has a small effect. A command that writes an installed prefix has a larger effect. A command that modifies systemd or SELinux would need a much stronger trust story and is not part of the current project authority posture.

Type	Allowed examples	Forbidden current examples
ReportOnly	latticra-seal report, dry-run evidence display.	Runtime process blocking.
VerifyLocal	Hash comparison, manifest presence check.	Network reputation lookup.
WriteEvidence	reports, plans, receipts, component markers.	Kernel module installation.
GuardedUserInstall	user-local prefix files and wrappers.	Root package install.
FutureRuntimeHandoff	metadata for a future decision engine.	Claiming sandbox enforcement today.

Chapter 10. Policy as a Fail-Closed Compiler

Policy is the subsystem that turns measured facts and requested effects into decisions. The correct default is fail closed.

A fail-closed policy does not silently pass malformed input, guess intent, grant authority, or claim enforcement. It emits a denial reason. This is exactly the behavior that makes Latticra valuable to a developer: failures are not merely red lights; they are explanations of why the state cannot be trusted or the claim cannot be made.

10.1 Policy input language

A mature policy layer should accept a small, explicit input language: manifest scope, authority vector, requested mode, measured evidence, lock status, command identity, and environment facts. That language should be strict enough to reject unknown authority modes and flexible enough to describe future runtime-boundary metadata without implying enforcement.

Definition 10.1 - Policy input and output sketch.

```
PolicyInput = {
    command: "seal.report",
    mode: "report-only",
    scope: ManifestScope,
    requested_authority: AuthorityVector,
    measured: [MeasuredFile],
    lock_present: Bool,
    manifest_present: Bool
}

PolicyOutput = Pass | Fail(reason) | Deny(reason)
```

10.2 Denial is a feature

In many systems, a denial is treated as a user experience failure. In Latticra, denial is part of the project promise. A policy-denial regression lane proves that unsafe or unsupported states are rejected. That makes denial a positive evidence artifact, not a nuisance.

Condition	Expected policy behavior	Example reason
Malformed manifest	Deny	manifest could not be parsed safely.
Path outside declared scope	Deny	path escapes manifest scope.
network_authority=true	Deny	network authority not implemented.
root_authority=true	Deny	root authority not allowed in current lane.
runtime enforcement claim	Deny	runtime enforcement not implemented or tested.
Changed hash	Fail or report change	current digest differs from lock baseline.
Missing lock	Fail or warn by mode	baseline unavailable for verification.

10.3 Policy and language design

A computer scientist can view Latticra policy as a static analysis over claimed effects. The analyzer has simple goals: reject unknown syntax, reject unsound authority, reject out-of-scope paths, and produce precise errors. This is the right direction because the project is not yet trying to be an all-knowing runtime guard; it is trying to make the static and local story correct.

Chapter 11. Reports, Receipts, and Witness Objects

The substrate is only as useful as the evidence it leaves behind. Latticra should treat every report, receipt, and manifest as a witness object with a schema.

A witness object is a record that lets another program or human reconstruct what was checked, when it was checked, where it was checked, which mode was used, what changed, what failed, and what authority was not used. This is more than logging. A log can be incidental. A witness object is designed.

11.1 Report semantics

A Seal report should be read as a bounded statement about the relationship between current local evidence and declared expectations. PASS means the checked evidence matched the command expectations. FAIL means evidence or policy did not match. DENY means the tool refused to proceed because the request or state was unsafe, unsupported, malformed, or out of scope.

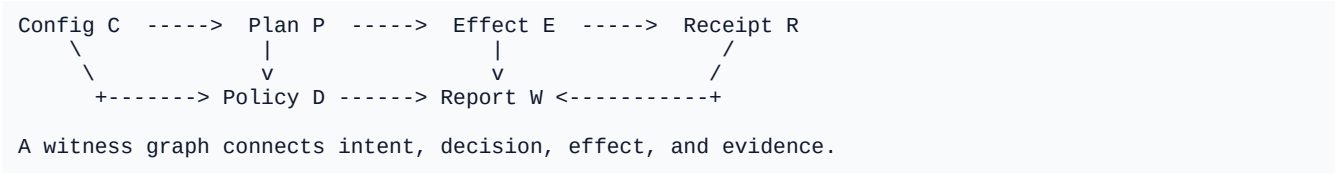
Status	Meaning	Non-meaning
PASS	Checked local evidence matched declared expectations.	Does not mean the host is secure.
FAIL	Evidence did not match or the check could not complete safely.	Does not necessarily imply compromise.
DENY	Policy rejected request or state before accepting it as valid.	Does not mean the project is broken.
WARN	A non-fatal issue should be read by the operator.	Does not grant authority.

11.2 Receipt semantics

Receipts should be constructed so that an auditor can answer two questions: what did the operator ask for, and what did the project actually do? The receipt should include config, plan, target prefix, result, installed paths, and authority flags. In a future signed-evidence model, receipts may also include signatures, key identifiers, or certificate chains, but those should not be claimed until implemented and tested.

11.3 Witness graph

Figure 11.1 - Witness graph.



The witness graph is useful because it generalizes across components. A Seal command has a manifest, a policy decision, a measurement effect, and a report. An installer action has config, plan, guarded writes, and a receipt. A future runtime handoff should have metadata, a capability gate, a decision, and a no-effect dry-run trace before any enforcement claim is allowed.

Chapter 12. Threats, Non-Goals, and Honest Boundaries

Honest boundaries are not marketing constraints; they are technical security controls against operator confusion.

A security project that overclaims creates risk. Operators may deploy it where it does not belong. Reviewers may assume protections that do not exist. Developers may build future layers on false premises. Latticra should treat unsupported public claims as a security bug.

12.1 Threat model

Threat	How Latticra responds now	Future direction
Project drift	Manifest/lock comparison and report evidence.	Signed baselines and richer schemas.

Operator confusion	Authority flags, receipts, explicit non-claims.	Stronger UI gating and claim linting.
Installer overreach	Guarded user-local prefix and root/network refusal.	Formal installer policy and reproducible packages.
Documentation overclaim	Boundary docs and claim ledger.	CI that checks public wording.
Runtime compromise	Not currently handled as enforcement.	Future runtime-boundary handoff research.
Supply-chain substitution	Hash baselines and receipts begin the story.	Cryptographic signatures and trust roots.

12.2 Non-goals

The current project should not be described as malware prevention, ransomware prevention, a sandbox, a kernel module, an SELinux policy system, a systemd enforcement system, a root installer, a network authority, a runtime enforcement authority, certified host protection, a Fedora-approved package, or an OS replacement. These are not stylistic cautions. They are correctness constraints.

12.3 The boundary rule

Rule 12.1 - Boundary rule.

```
No claim without evidence.
No evidence without test.
No test without documentation.
```

This rule should remain visible in the repository, the Panel, and the handbook. It is the clearest expression of Latticra as a substrate for modern security: the system is allowed to grow, but every growth step must leave a trail.

Part IV - Implementation Workbook

The book now turns from architecture to practical engineering patterns that can guide future Latticra work.

Chapter 13. Command Contracts

Every command in Latticra should have a contract that a reviewer can understand without reading all of the source code.

The Seal handbook already states the right documentation posture: every command should say what it reads, what it writes, what it refuses to do, whether it is report-only, whether it requires root, whether it uses the network, what counts as PASS, and what counts as FAIL. This chapter turns that into a reusable command contract template.

Template 13.1 - Latticra command contract.

```
# Command contract template
name:
summary:
mode:
reads:
writes:
refuses:
authority:
  root: false
  network: false
  runtime_enforcement: false
  kernel_modification: false
  systemd_modification: false
  selinux_modification: false
exit_codes:
  0: pass
  1: fail
  2: policy_denial_or_bad_input
pass_meaning:
fail_meaning:
denial_meaning:
reports_or_receipts:
examples:
```

13.1 Example: seal report

Field	Value
name	latticra-seal report
mode	report-only
reads	local manifest, lock material, selected local files, OS facts
writes	report output only, if configured
refuses	network authority, root authority, runtime enforcement authority
PASS	checked local evidence matched declared expectations
FAIL	missing/malformed evidence, mismatch, or safe completion failure

13.2 Example: installer dry-run

Field	Value
name	make -C installer dry-run
mode	dry-install metadata
reads	installer config, component manifest, local repo files
writes	plan and receipt artifacts
refuses	host mutation, root authority, network authority
PASS	plan/receipt generated and authority constraints preserved

FAIL	invalid config, missing scripts, unsafe prefix, or tooling error

13.3 Command review checklist

- Can the read set be named without vague phrases such as "system state"?
- Can the write set be inspected and removed?
- Can a policy denial be reproduced with a minimal test fixture?
- Does the command use the network, directly or indirectly?
- Would a reasonable user infer a stronger security claim than the command proves?
- Does the README wording match the command contract?

Chapter 14. Schemas and Formats

Latticra should prefer small, stable schemas over prose-only state. Schemas are how future tools can consume today's evidence.

The first schema family should cover manifests, reports, receipts, policy-denial reasons, and runtime-handoff metadata. The schemas should be simple enough for shell and Rust tooling, but strict enough to reject ambiguity. TOML, JSON, and line-oriented key/value formats can all be useful, provided the parser behavior is documented.

14.1 Manifest schema requirements

- Schema version and project identifier.
- Mode: report-only, verification, or future explicitly gated modes.
- Scope: include/exclude path lists with normalized semantics.
- Authority vector: explicit booleans for all sensitive authorities.
- Policy requirements: fail-closed behavior and denial classes.
- Report behavior: output path, format, and human-readable summary behavior.

14.2 Report schema requirements

- Timestamp, host facts, mode, prefix, command identity.
- Manifest and lock presence plus validation status.
- Hash counts: checked, changed, missing, new, ignored.
- Authority flags with zero values preserved, not omitted.
- Warnings and failures as structured arrays with stable codes.
- Final status and explicit PASS/FAIL/DENY meaning.

14.3 Denial code taxonomy

Code	Meaning
DENY_UNKNOWN_AUTHORITY	An unsupported authority key or mode was requested.
DENY_NETWORK_AUTHORITY	Network authority was requested in a lane that has none.
DENY_ROOT_AUTHORITY	Root authority was requested in a lane that has none.
DENY_RUNTIME_ENFORCEMENT	Runtime enforcement was requested or claimed without implementation.
DENY_SCOPE_ESCAPE	A path resolves outside declared scope or guarded prefix.
DENY_MALFORMED_MANIFEST	Manifest could not be parsed or lacks required fields.
DENY_UNSUPPORTED_CRYPTO_CLAIM	A signing/encryption claim is not backed by implemented

	verification.

14.4 Runtime-handoff metadata

Future runtime-boundary handoff metadata should be designed before enforcement exists. The metadata can describe decisions in no-effect dry-run form: what would be denied, why, under which capability gate, with which evidence. The handoff schema must not imply that a runtime enforcement path exists until it does.

Definition 14.1 - Future no-effect runtime handoff sketch.

```
RuntimeHandoffDryRun = {
    schema_version: 1,
    mode: "no-effect-dry-run",
    capability_gate: "future-runtime-boundary",
    input_report: "path-or-digest",
    decision: "would-deny" | "would-allow" | "inconclusive",
    reason_codes: [String],
    enforcement_performed: false
}
```

Chapter 15. Test Lanes and Continuous Evidence

Tests are not just quality gates in Latticra. They are the mechanism that keeps claims from exceeding evidence.

The current project already includes or references useful lanes: Seal smoke checks, policy-denial checks, installer dry-run, guarded local install verification, uninstall verification, Rust formatting/checks for Panel installer code, shell script validation, Fedora VM CLI/RPM validation, and CI status tracking. The next step is to turn these into a continuous evidence architecture.

Lane	Purpose	Evidence expected
make seal	Smoke-check local Seal behavior.	Report or command result.
make seal-policy-denials	Prove unsafe policies are denied.	Denial transcript and status.
installer dry-run	Validate plan without host mutation.	Plan and dry-run receipt.
verify-local	Check guarded user-local install.	Installed paths and Seal report generation.
uninstall-local	Remove managed files.	Removal transcript and absence checks.
Panel Rust check	Validate GUI installer crate builds.	cargo fmt/check output.
Fedora VM/RPM lane	Research local packaging and validation.	VM transcript with non-claims preserved.

15.1 Evidence-level testing

A normal test says "the code behaved." An evidence-level test says "the code behaved, emitted the expected witness, and did not claim an unsupported authority." For Latticra, that second form is the more important one.

Example 15.1 - Evidence-level assertions.

```
assert report.status == "PASS"
assert report.network_authority == 0
assert report.root_authority == 0
assert report.runtime_enforcement_authority == 0
assert "host is secure" not in report.public_summary.lower()
```

15.2 Claim linting

A future CI lane should lint README files, docs, Panel text, and release notes for forbidden phrases such as production-ready, ransomware-proof, malware-proof, kernel-enforced, runtime-enforced, certified, or guaranteed secure. The linter should fail unless the claim ledger says the corresponding evidence and tests exist.

15.3 Disposable validation environments

Host-level validation should remain isolated and explicit. Disposable Fedora VM lanes are useful because they separate validation from daily-driver risk. They also allow the project to say exactly what was validated without claiming Fedora approval, production readiness, or immutable-host compatibility.

Chapter 16. Roadmap and Research Agenda

The roadmap should grow in disciplined layers. Each phase adds a stronger substrate property without skipping evidence.

The original Seal roadmap is a good foundation: documentation baseline, CLI clarity, manifest and lock hardening, report schema, Panel integration, cryptographic strengthening, runtime-boundary handoff, and Fedora/Linux integration research. The whole-project roadmap should use the same layering but apply it across Latticra components.

Phase	Goal	Done when
1. Documentation baseline	Make the project understandable.	Every component has role, status, non-claims, and command map.
2. CLI clarity	Make commands predictable.	Read/write/refuse/root/network/PASS/FAIL contracts exist.
3. Manifest and lock hardening	Make local verification reproducible.	Schema, lock format, update rules, ignored paths, changed output, malformed tests.
4. Report schema	Make evidence human- and machine-readable.	Required fields, warning/failure classes, stable status values.
5. Panel integration	Make Seal and installer evidence visible.	Panel exposes report-only scan, plans, receipts, manifest/lock state, no authority escalation.
6. Cryptographic strengthening	Move from hashes to signed evidence.	Signing model, key handling, trust roots, invalid signature tests.
7. Runtime-boundary handoff	Prepare metadata without overclaiming.	No-effect dry-run handoff, capability gate, denial tests.
8. Fedora/Linux integration research	Explore host integration carefully.	User-local evidence, no-root assumptions, RPM boundaries, SELinux/systemd non-claims.

16.1 Research questions

- What is the smallest manifest schema that can support useful reproducible evidence?
- Can public claim linting be tied directly to the authority vector and test results?
- What receipt format is expressive enough for local installs but simple enough for shell tooling?
- How should a future trust root be introduced without weakening the current no-claim posture?
- What runtime-boundary metadata can be generated today that remains honest about no enforcement?
- Can Panel UX make authority boundaries intuitive without hiding the formal model?

16.2 The long-term shape

The long-term Latticra system should be a substrate where every artifact is typed by its effects, every security-relevant action leaves a witness, every public claim is checked against evidence, and every future authority upgrade passes through a capability gate. That is a meaningful contribution to modern security even before it becomes a production enforcement system.

Appendices

Reference material for developers, reviewers, and project writers.

Appendix A. Glossary

A compact vocabulary for the Latticra substrate.

Term	Meaning
Authority vector	A structured record of which powers a command or lane has or explicitly lacks.
Boundary	A limit on scope, authority, or claim strength.
Claim ledger	The project record of what is implemented, evidenced, planned, or forbidden to claim.
Effect	An observable action such as reading, writing, installing, networking, or enforcing.
Evidence-bound	Constrained by reproducible evidence and tests.
Fail closed	Reject unsafe, malformed, unsupported, or contradictory input instead of guessing.
Manifest	A declaration of inspection scope and policy expectations.
Receipt	A witness for an operator action or installer effect.
Report-only	A mode that inspects and describes local state without runtime enforcement.
Runtime handoff	Future metadata path for enforcement decisions, not current enforcement.
Seal	The Latticra evidence, reporting, and policy-boundary layer.
Substrate	A foundational layer of state, policy, evidence, and authority relations.

Appendix B. Reviewer Checklists

Use these checklists when reviewing pull requests, release notes, or public project pages.

B.1 Documentation checklist

- Does the text distinguish evidence from enforcement?
- Does it preserve current non-claims around root, network, runtime, kernel, systemd, SELinux, and production readiness?
- Does every command document read/write/refuse behavior?
- Are PASS and FAIL meanings bounded and precise?
- Are future features labeled as future work rather than current behavior?

B.2 Implementation checklist

- Does the code normalize paths before comparing scope?
- Does malformed policy fail closed?
- Does a report or receipt include authority flags even when all are zero?
- Does a dry-run avoid host mutation?
- Does a local install stay inside the guarded prefix?
- Are denial reasons stable enough for regression tests?

B.3 Panel UX checklist

- Does the UI show whether the current action is dry-run or guarded install?
- Does it make authority visible before an action?
- Does it show plan and receipt paths?
- Does it avoid language that implies production protection?
- Does it distinguish crypto posture selection from actual signing/encryption behavior?

Appendix C. Public Wording Guide

Project language should be rigorous enough that a reader can infer the implementation boundary.

Use this kind of wording	Avoid this kind of wording
early verification and reporting layer	production security platform
local evidence and policy-boundary subsystem	malware or ransomware prevention
report-only state inspection	runtime enforcement
manifest/hash baseline mechanism	kernel-enforced protection
guarded user-local installer	root installer
future runtime-boundary research	sandbox today
policy-regression-backed denial behavior	guaranteed secure

A strong public sentence: "Latticra is an early, local, evidence-bound substrate for report-only verification, policy regression, manifest/hash baselines, guarded user-local installation, and future runtime-boundary research."

A sentence to reject: "Latticra protects production systems from malware with kernel-enforced runtime isolation." That sentence exceeds current evidence.

Appendix D. Source Basis for This Edition

This section records how this draft was grounded.

This edition was derived from the uploaded Latticra Seal handbook and broadened with recent File Library material containing Panel installer notes, dry-run and local-prefix install transcripts, receipt and component-marker evidence, Fedora validation snippets, and the one-pass field-engine browser artifact. The expansion is conservative: it treats the broader project as a substrate architecture and does not claim production security readiness, runtime enforcement, network authority, root authority, kernel integration, SELinux integration, systemd enforcement, or Fedora approval.

Source area	Used for
Seal handbook	Seal role, architecture, policy, reports, boundaries, and roadmap.
Panel installer material	Project-level component map, UI/installer workflow, authority visibility.
Installer receipts and dry-run evidence	Guarded user-local installation and receipt semantics.
Fedora/RPM validation snippets	Validation lane framing with explicit non-claims.
One-pass field engine	Research artifact and witness-manifest pattern.

Future editions should replace inferred project-level connective tissue with canonical repository documents as they mature: a root architecture spec, Lat/LIR spec, Panel UX contract, installer readiness contract, report schema, receipt schema, and runtime-handoff specification.

Quick Start: How to Read Latticra as a Computer Scientist

1. Start with the authority vector. Identify what the tool is allowed to do and what it explicitly cannot do.
2. Read the manifest and scope rules. A local evidence tool is only meaningful if its observation set is defined.
3. Inspect the report or receipt. Look for timestamp, mode, prefix, authority flags, warnings, failures, and status meaning.
4. Review denial tests. A good policy layer proves that unsafe states do not pass.
5. Compare documentation to evidence. Delete or weaken any sentence that outruns implementation and tests.
6. Treat future runtime-boundary work as metadata research until an enforcement path exists and is tested.

The result is a project that can mature without pretending. That is the Latticra contribution: not a premature claim of security, but a substrate where the effects of modern security work become explicit enough to verify.